Trade Study: Python vs. C++

- **Decision:** Mixed Python/C++26 development with incremental C++ adoption.
- **Status:** Preliminary recommendation · September 13, 2026.

Problem: The vehicle needs to detect, track, and follow an object using camera, LiDAR, and IMU observations. The existing hardware studies favor components with established software support and straightforward integration. Our software approach should similarly let the team demonstrate the basic design quickly while supporting more ambitious perception, estimation, and control later. This study compares Python-led development, C++-led development, and a mixed approach that progressively introduces custom C++ where performance or capability needs justify it. All three can reuse existing native libraries and GPU acceleration. [1]

Project context: Jetson Nano is the working target. The majority of the team is more comfortable with Python; many members have explicitly reported limited or no C++ experience. The initial object-following pipeline is inferred from the hardware studies. A specific detector, middleware, and performance budget have not yet been established.

Rankings: Scores are engineering judgments, not measured speedups. Grades use a ten-point scale: 2 = poor fit, 6 = adequate fit, 10 = strong fit. Weights reflect both initial delivery and future development; weighted score = sum of weight * grade.

		Python-led	C++-led	Mixed Python/C++
Criteria	Weights	Grade	Grade	Grade
Time to demonstration	0.30	10	2	10
Team accessibility	0.25	10	2	8
Runtime performance control	0.20	6	10	10
Algorithm customization	0.15	6	10	10
Build/debug simplicity	0.10	8	6	4
Average Score	-	**8.00**	**6.00**	**8.40**
Weighted Score	**1.00**	**8.40**	**5.20**	**8.90**

Delivery and accessibility receive 55% of the weight because most contributors have little C++ experience. Performance and customization receive 35%; build/debug simplicity receives 10%. Both Python-led and mixed approaches score 10 for demonstration time: custom C++ can wait until after the first milestone. Mixed development's lower accessibility and simplicity grades account for later native work.

Decision sensitivity: Moving 0.15 of weight from performance control to demonstration speed produces Python-led **9.00**, mixed **8.90**, and C++-led **4.00**. A demonstration-only priority favors Python-led development; the mixed approach preserves future native options with the same initial delivery path.

Breakdown

1st Choice: Mixed Python/C++

With a weighted score of **8.90**, the mixed approach best balances initial delivery and long-term flexibility. The first milestone can use Python throughout, alongside existing libraries. Introduce custom C++ behind stable interfaces when a stage needs lower latency, less memory, or new behavior. Python can continue supporting configuration, experiments, and analysis.

Stable interfaces and replayable data make replacement practical; native packaging and cross-language debugging add maintenance cost.

2nd Choice: Python-led

Python-led development scores **8.40**, providing the most familiar development path for the majority of the team. Optimized libraries already execute substantial work in native code: TensorRT provides both Python and C++ APIs. Rewriting calls to the same engine does not establish a GPU speedup. [2]

Its weaker scores concern custom processing and execution control. Native work becomes appropriate when Python loops, conversions, or scheduling measurably limit the application. Components that meet requirements can remain Python.

3rd Choice: C++-led

C++-led development scores **5.20** and is not recommended for the first milestone. Requiring most application work in C++ would make delivery depend on substantial onboarding and a small number of experienced contributors, earning 2 for both demonstration time and accessibility. Its strong technical grades recognize the later value of custom engines with compact storage, reusable buffers, and specialized algorithms.

Data-oriented design and performance

Data-oriented design (DoD) organizes storage and processing around the workload. Replacing scattered objects and pointer traversal with suitable contiguous records or arrays can reduce memory traffic and improve batch processing. These changes can deliver order-of-magnitude gains when memory behavior dominates. Colfax measured a **30x performance penalty** from false sharing in single-precision particle binning on a dual-socket Xeon; reorganizing and padding storage recovered nearly all reference performance. This is a layout/contention result, not a Python-versus-C++ or graph benchmark. ISO's performance report separately explains compact storage's traversal advantages. [5]

CPython lists contain object references, whereas typed NumPy arrays can hold packed numeric data. Python can therefore retain the benefits by passing batches to the same efficient native engine, with conversion and call overhead measured separately. C++ makes custom layouts practical, but does not guarantee them. Graph layout, traversal order, mutation costs, and workload size determine the gain, which must be measured for our workload. [5]

Compatibility and transition

Python can import native modules through **pybind11** or **nanobind**. Concepts and templates do not require a virtual-class hierarchy: `Tracker<AssociationPolicy, MotionModel>` can expose a compiled specialization as a Python `Tracker`. New template combinations require a rebuild; tuning parameters can remain runtime values. [3]

Pass complete frames, scans, or batches through compatible buffers with explicit type, layout, lifetime, and synchronization rules. Release Python's interpreter lock around native work that does not access Python objects. [4]

Development approach

C++ proficiency and maintainability

Limited C++ experience increases the risk of lifetime errors, undefined behavior, and inconsistent designs. Create a concise **C++ development guide** covering explicit ownership, automatic resource management, borrowing, and error handling. Experienced contributors should support onboarding, pairing, and review; compiler warnings and sanitizers should help catch mistakes. Use templates and concepts where they simplify the design. These practices reduce risk and spread expertise, avoiding dependence on one person. [6]

Libraries, portability, and customization

CUDA and TensorRT provide useful NVIDIA acceleration but introduce vendor and deployment dependencies; choosing C++ retains those dependencies when it uses the same APIs. PyTorch is broader and supports other backends, including ROCm. Keep vendor-specific implementations behind replaceable interfaces. Existing tools establish a baseline against which custom work may be better or worse in speed, resource use, capability, or maintenance cost. CPU optimization also depends on hardware; GPU kernels additionally require backend/device-specific tuning and attention to transfers and synchronization. [7]

Profile before changing the implementation

Use profiling to locate hotspots and identify whether time is spent computing, allocating, copying, waiting, or traversing data. For Python, start with `cProfile`; use native profiling inside C++ and a framework/GPU profiler where appropriate. Use separate benchmark runs to compare implementations: Python's profiling instrumentation can distort Python-versus-native comparisons. Apply DoD to the measured bottleneck, then verify tracking behavior, end-to-end p95/p99 latency, throughput, and memory. [8]

Version the prototype and add replay incrementally

Build and validate the first prototype on the vehicle, then tag its source revision and record dependencies, model, configuration, and calibration. Capture sensor recordings where straightforward, without making a replay framework a prerequisite for the first milestone.

As the design evolves, add the capability to run processing stages against recorded sensor inputs for regression tests and experiments. Version recordings and interfaces so alternatives can be compared with the baseline. Replay complements vehicle testing; physical behavior and target-device performance still require the hardware. Use results and available components to guide improvements, preserve rollback, and retain useful Python tooling as native processing expands.

Sources:

1. Project vehicle, camera, LiDAR, and IMU trade studies.
2. NVIDIA: TensorRT Python and C++ capabilities.
3. pybind11: binding concrete template instantiations.
4. pybind11 array interfaces, GIL handling, and nanobind array exchange.
5. ISO C++ performance report, p. 47; Colfax layout benchmark, §6.2; CPython lists; NumPy storage.
6. C++ Core Guidelines: resource management and interfaces.
7. TensorRT compatibility; PyTorch ROCm support.
8. Python profiling and measurement caveats; PyTorch profiler.